

Monorepo & Shared Packages Specification

Turborepo, PNPM Workspace, Shared Schemas, Types, Utils, and CI/CD Pipelines

MONOREPO SPEC

Project: RVCC Enterprise
Monorepo

Version: 2.0.0
(Production)

Architecture: Next.js 16 + Edge +
Prisma

Status: Verified &
Active

1. Executive Summary & Ecosystem Overview

The **RVCC Enterprise Digital Platform** is an enterprise-grade, monorepo-based cloud ecosystem engineered to handle end-to-end corporate branding, multi-tier supplier qualification, real-time blind bidding, internal requisition governance, and multi-tenant administrative oversight. It is structured as an optimized Turborepo monorepo powered by PNPM workspaces, Next.js 16 (App Router with Turbopack), Cloudflare Edge Runtime with Prisma ORM, and automated CI/CD pipelines.

Core Business Capabilities

- **Corporate Public Portal:** 3D WebGL showcase, career applications, dynamic flipbook documentation.
- **Vendor Onboarding Train:** 10-step audited registration with document OCR verification.
- **Real-Time Blind Bidding:** SSE sub-second quote stream with automated L1 ranking.
- **Procurement Governance:** Departmental requisition workflows and cost tracking.

Technical Standards & Metrics

- **Monorepo Codebase:** 5 Applications + 3 Shared Packages.
- **Total Scope:** 580 production source files, 65,453 lines of clean TypeScript.
- **Database Entity Model:** 31 Relational Prisma Models + 10 Business Enums.
- **Testing & Quality:** 10/10 passing Vitest suites, ESLint 9 Flat configs, Prettier.

2. Monorepo Structure & Workspace Topology

The workspace is orchestrated using `pnpm-workspace.yaml` and `turbo.json` for maximum build parallelization, cache deduplication, and strictly isolated package boundaries.

```
RVCC-MONOREPO/
├── apps/
│   ├── web/           # Port 3000 - Public Corporate & Vendor Onboarding Application (Next.js 16)
│   ├── admin/         # Port 3001 - Enterprise Super-Admin & Sourcing Governance ERP (Next.js 16)
│   ├── vendor/        # Port 3002 - Vendor Portal & Real-Time Blind Bidding Cockpit (Next.js 16)
│   ├── procurement/   # Port 3003 - Internal Department Requisition & Approval Engine (Next.js 16)
│   └── api/           # Port 8787 - Edge Backend API, SSE Streaming Engine & Prisma ORM
├── packages/
│   ├── schemas/       # @rvcc/schemas - Shared Zod validation contracts & domain types
│   ├── types/         # @rvcc/types - TypeScript interfaces, DB entity shapes & API contracts
│   └── utils/         # @rvcc/utils - Common currency math, string sanitizers & formatters
└── .github/workflows/ # Automated CI/CD test, build, and deploy pipelines
```

3. Shared Packages Architecture

Package Name	Key Exports & Modules	Responsibility & Consumption
@rvcc/schemas	auth.ts , procurement.ts , sourcing.ts , vendor-onboarding.ts , rbac.ts , audit.ts , enums.ts	Single source of truth for runtime validation using Zod. Consumed across API endpoints and frontend forms for end-to-end type safety.
@rvcc/types	Database entity types, API response generics (ApiResponse<T>), Live Bid payload contracts, Currency conversion shapes.	Compile-time type contracts ensuring strict interface compliance between backend workers and frontend applications.
@rvcc/utils	formatters.ts , currency.ts , sanitize.ts , rank.ts , cn.ts	Shared utility functions for currency formatting, SAR normalization, class merging (Tailwind Merge + Clsx), and ranking algorithms.

4. Turborepo Pipeline & Build Cache Matrix

The Turborepo pipeline (`turbo.json`) enforces topological build ordering and smart caching across the 8 monorepo packages:

```
{
  "$schema": "https://turbo.build/schema.json",
  "tasks": {
    "build": {
      "dependsOn": ["^build"],
      "outputs": [".next/**", "!next/cache/**", "dist/**"]
    },
    "test": {
      "dependsOn": ["^build"],
      "inputs": ["src/**/*.ts", "src/**/*.tsx", "vitest.config.ts"]
    },
    "lint": {
      "dependsOn": ["^build"]
    }
  }
}
```

5. CI/CD & DevOps Pipeline Architecture

Continuous Integration and Deployment is automated via GitHub Actions with distinct pipelines for quality assurance and production releases.

CI Workflow (`ci.yml`)

- **Trigger:** Pull Requests and pushes to `dev` branch.
- **Steps:** Dependency caching, Prisma schema validation, TypeScript check, ESLint verification, Vitest automated unit testing.
- **Guarantees:** Zero broken builds or type errors can reach production.

CD Workflow (`deploy.yml`)

- **Trigger:** Pushes and tags on `main` branch.
- **Automated Deployments:** Cloudflare Workers deployment for `apps/api` via Wrangler, Vercel deployments for `web`, `admin`, `vendor`, and `procurement`.
- **Rollback:** Instant zero-downtime rollback capabilities.

6. Code Quality, Formatting & Tooling Standards

- **ESLint 9 Flat Configuration:** Standardized flat configs across all packages with `eslint-plugin-unused-imports` (automatic pruning of unused imports) and `eslint-plugin-simple-import-sort` (deterministic import ordering).
- **Prettier & Tailwind Plugin:** Enforces consistent indentation, line wrapping, and automatic alphabetical ordering of Tailwind CSS utility classes via `prettier-plugin-tailwindcss`.
- **TypeScript 5.8+:** Strict mode enabled with isolated modules, no implicit any, and strict null checks across the monorepo.
- **PWA Lifecycle Management:** Automatic service worker version bumping during prebuild scripts ensuring clients receive instant asset cache refreshes on new releases.